

Nexus Shield — AI Agent Action Governance & Verification Platform

Know what your agents are allowed to do. Stop what they shouldn't. Prove what actually happened.

Executive architectural overview · CISO / CTO briefing

<https://www.nexusshield.ai/> · Confidential — for stakeholder review

Introduction

Official site: <https://www.nexusshield.ai/>

Live dashboard: <https://nexus-shield-dashboard.vercel.app>

Document class: CISO / CTO / executive briefing · architecture overview

Generate PDF: `python scripts/generate_enterprise_deck.py --lang en` · Turkish: `ENTERPRISE_PITCH_AND_VISION_TR.md (--lang tr)`

1. Executive Summary & Core Positioning

Official name

Nexus Shield — AI Agent Action Governance & Verification Platform

Core tagline

Know what your agents are allowed to do. Stop what they shouldn't. Prove what actually happened.

Infrastructure promise

Your AI agents. Your infrastructure. Your data. Your policies.

Positioning shift

Legacy framing (deprecated)	Nexus Shield framing
Generic “AI security”	Agent action governance at the tool execution boundary
Prompt-only firewall	Intent verification + policy decision on every tool call
Log retention	Universal Action Receipts (UAR) with SHA-256 evidence_hash
Opaque block rates	Transparent Proof Center — benchmark lane vs production UAR ledger

Upstream LLM inspection, PII redaction, and router guardrails remain supporting Security Engines. The primary product object is the UAR — a cryptographically sealed record of what was attempted, what policy decided, and what state changed.

Executive outcomes

- Govern: Define what autonomous agents may execute against CRMs, databases, payment APIs, and MCP tool surfaces.
- Stop: Enforce BLOCK, adaptive READ_ONLY, REQUIRE_APPROVAL, or fleet kill switch without guessing from chat logs alone.
- Prove: Export SIEM-ready evidence, verify receipts on `/verify`, and satisfy audit requests with reproducible hashes — not screenshots.

Definitive product clarification (CISO / CTO)

Nexus Shield is not a generic “AI security” product or an LLM firewall that only scores prompt safety. It is an Agent Action Governance & Verification Platform built for the moment tools execute:

- Tool execution control — every side-effecting MCP / API call is evaluated before it runs.
- Parameter hijacking — detect when tool names or arguments diverge from declared intent.
- Intent divergence — quantify misalignment (e.g. 96%) and enforce graded policy instead of blind trust in the model transcript.

Prompt filtering may reduce chat-layer risk; it does not replace governance at the action boundary. That boundary is where legal exposure, data loss, and operational paralysis actually occur.

Air-gapped & self-hosted trust guarantee

For security leaders evaluating vendor risk: Nexus Shield’s data plane (nexus) is designed air-gapped and self-hosted by default.

Commitment	Detail
Default posture	NEXUS_AIRGAP=true, NEXUS_CLOUD_CONNECT=false in deployments/
Data residency	Agent prompts, tool payloads, policies, and UAR ledger stay inside your VPC, cluster, or on-prem boundary
Optional cloud	Control plane (nexus-control) is off by default — license/signature sync only when you explicitly enable it
Deploy anywhere	Docker Compose or Helm on AWS, Azure, GCP, or private data centers — your API base URL, not ours

CISO objection answered: “Will our agent data leave the corporate boundary?” — No, when you run the self-hosted data plane with air-gap defaults. Governance and cryptographic proof are local; egress is a conscious opt-in, not a product requirement.

2. Before & After — Real-World Scenario (Core Pain Killer)

This is the operational contrast security leaders ask for in the first five minutes of review.

Before Nexus Shield — today’s risk

A production LLM agent wired through LangChain, CrewAI, or custom orchestration receives prompt injection or planner misalignment. It proposes a destructive or exfiltration tool call — for example:

- Unauthorized database write or schema change via an admin tool.
- Bulk customer export or webhook callback exfiltration through an MCP server.

The tool runs. After the incident, enterprise telemetry shows only that the “LLM generated a response.” There is no cryptographically verifiable link between declared user intent, requested tool, policy decision, and before/after system state. Legal and compliance teams lack defensible evidence; operations teams freeze or kill entire agent fleets because they cannot prove what was attempted vs. what was blocked.

With Nexus Shield — tomorrow’s control

The data plane intercepts the proposed call at the execution boundary (POST /api/v1/action/evaluate on the dashboard path, or self-hosted POST /v1/intercept):

1. Intent declared — e.g. “Read-only invoice summary.”
2. Tool proposed — e.g. `export_customer_database` with exfil-shaped parameters.
3. Intent divergence detected — e.g. 96% misalignment, violations such as `INTENT_ACTION_DIVERGENCE`.
4. Policy enforced — automatic `BLOCK`, or `READ_ONLY` / `REQUIRE_APPROVAL` instead of shutting down every agent.
5. UAR sealed — immutable Universal Action Receipt with SHA-256 `evidence_hash` for audit, SOC 2 / ISO, and regulator-ready defense.
6. Verify & export — `/verify`, local JSONL ledger, SIEM JSONL — proof of what happened, not a chat log guess.

Dimension	Before Nexus Shield	With Nexus Shield
Visibility	“Model replied”	Intent → tool → decision → state → UAR
Response	Fleet kill or hope	<code>BLOCK</code> , <code>READ_ONLY</code> , approval queue
Audit defense	Screenshots & anecdotes	SHA-256 <code>evidence_hash</code> , reproducible verification
Data boundary	Unclear vendor egress	Self-hosted, air-gapped data plane (default)

3. Evolution & Development Journey

Phase 1 — LLM guardrails (foundation)

Early work focused on prompt injection, PII leakage, and basic LLM proxy patterns. Valuable for chat-layer risk, but insufficient when agents gained tool execution and persistent credentials.

Phase 2 — Enterprise pain discovery (agentic boundary)

Customer and red-team feedback converged on a different class of risk:

- Tool misuse and MCP server hijacking (LangChain, CrewAI, custom agents).
- Privilege escalation via chained tool calls.
- Intent / action divergence — stated user goal vs. proposed destructive or exfiltration tool.
- Operational dilemma: hard-kill the entire agent vs. `READ_ONLY` degradation vs. human-in-the-loop.

Nexus Shield pivoted from “filter prompts” to govern actions.

Phase 3 — Harness vs runtime decoupling

The open-source `nexus-harness-benchmark` (`harness/`) was explicitly separated from the production Action Firewall:

- Harness = reproducible scores, CVE-style presets, MCP-SEC-SCORE leaderboards.
- Runtime = live interception, tenant RBAC, local UAR ledger.

See `BENCHMARK_VS_ACTION_FIREWALL.md`.

Phase 4 — Enterprise readiness (cryptography & planes)

Milestone	Capability
cc20ed6044	Data plane packaging (Docker / Helm), UAR evidence chain, SIEM compliance JSONL, deployment docs
0d16526309	Product positioning as AI Agent Action Governance & Verification; canonical UAR_SCHEMA.md
4b46946548 / ef22090ffa	Proof Center copy transparency — evidence bundles per evaluated trajectory, not “blocked actions only”

Today: data plane (nexus) runs air-gapped by default; control plane (nexus-control) is optional for license and signature sync.

4. Enterprise Problem & Market Target

Target market

Segment	Why Nexus Shield
Financial services	Wire transfers, bulk export tools, SOX / PCI audit trails
Healthcare	PHI access via agent tools, minimum-necessary enforcement
Enterprise SaaS	Multi-tenant agents touching customer CRM / billing APIs
Autonomous DevOps	CI agents with production database or cloud control-plane tools

Problems solved

1. Unauthorized tool execution — agents invoking write/delete/export tools outside policy.
2. Intent divergence — planner or injected context diverges from declared business intent.
3. Missing audit trail — no cryptographic link between intent, tool, decision, and state.
4. Binary incident response — replace “shut down the agent” with graded enforcement (READ_ONLY, approval queues, scoped revocation).
5. Compliance evidence gap — SOC 2 / ISO 27001 / GDPR-KVKK requests need exportable, verifiable artifacts.

5. Repository Ecosystem (Open Source & Enterprise Stack)

Codename	Path / package	Role
nexus	enterprise/data_plane_api.py, policy engine, uar_store.py	Data plane — governance, UAR sealing, local ledger, POST /v1/intercept
nexus-control	enterprise/cloud_panel.py, tenant_manager.py	Control plane (optional) — orchestration, RBAC, SIEM export, license sync when NEXUS_CLOUD_CONNECT=true
nexus-agent-sdk-python	packages/python (pip install nexus-shield)	Python SDK — evaluate actions, map capabilities, embed interception
nexus-agent-sdk-bridge	packages/npm	Node / Vercel AI SDK bridge — MCP and HTTP proxy integration patterns
nexus-harness-benchmark	harness/	Evaluation only — 500+ MCP attack scenarios, multi-agent graphs, reproducible JSON evidence

Supporting surfaces:

- `nexus-shield-dashboard/` — Proof Center UI, Trust Hub, `/demo`, `/verify`, `/scan`.
- `presets/` — Deterministic CVE-style vulnerability demonstrations.
- `deployments/` — Docker Compose and Helm for on-prem / air-gap.

6. Site Modules & Interactive Funnel

Public and authenticated modules on the dashboard (see `nexus-shield-dashboard/lib/dashboard-nav.ts`):

Module	Route	Purpose
Setup Guide	<code>/dashboard</code>	Onboarding, integration checklist
Free Scan	<code>/scan</code>	Attack → Prove → Install → Protect — scan agent endpoints, MCP configs, GitHub patches for excessive authority
Agents	<code>/dashboard/agents</code>	Fleet visibility, capability maps
Action Firewall	<code>/dashboard/actions</code>	Live policy view, interception history
Threat Intel	<code>/dashboard/threat-intel</code>	Collective attack patterns, immune memory
Red Teaming	<code>/dashboard/simulator</code>	Synthetic attack vectors, simulator
Proof Center	<code>/proof-center</code>	Public harness metrics + methodology links
Trust Hub	<code>/dashboard/trust-hub</code>	Governance audit trail, receipt verification
Compliance	<code>/dashboard/compliance</code>	SOC 2 / ISO oriented reporting hooks, KVKK/GDPR-oriented evidence export
Challenge Engine	<code>/challenge</code> (API: <code>/api/challenge/evaluate</code>)	7-level attack sandbox and community leaderboard
Detect & Demonstrate	<code>/demo</code>	CVE presets, independent <code>/verify</code> URLs
Investor metrics	<code>/investor</code>	Growth and benchmark transparency

Core runtime APIs

Endpoint	Plane	Description
POST /api/v1/action/evaluate	Dashboard / managed API	Real-time tool-call evaluation → UAR envelope
POST /v1/intercept	Self-hosted data plane (:8090)	Same governance semantics on customer infrastructure
POST /api/v1/agent/trust	Dashboard	Agent reputation / MCP-SEC-SCORE updates after governed events
GET /verify	Public	Receipt hash presence check (receipt_hash, receipt_id)

7. Deployment Architecture — Self-Hosted, Air-Gapped & Custom Cloud

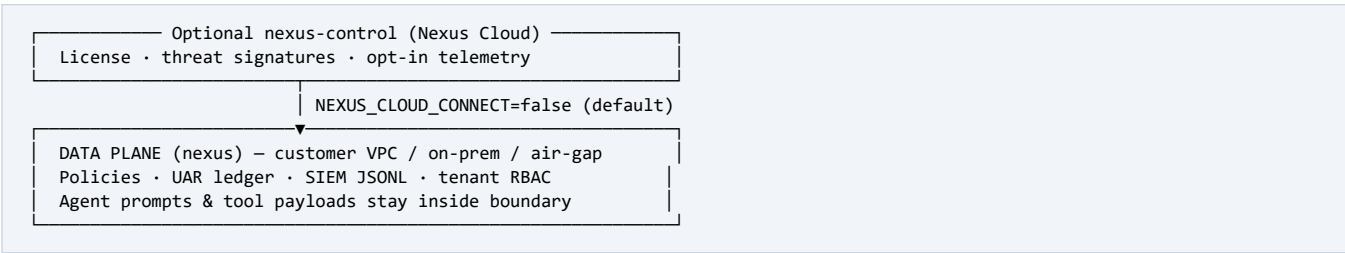
CISO trust guarantee (architecture)

Nexus Shield separates control from proof:

- Data plane (nexus) — mandatory for governance: interception, UAR sealing, RBAC, SIEM export. Runs entirely inside your environment with `NEXUS_AIRGAP=true`.
- Control plane (nexus-control) — optional Nexus Cloud peering for license and signature updates; not required for blocking, receipts, or compliance logs.

Agent prompts, tool arguments, and API payloads processed by the data plane are not sent to Nexus Shield SaaS when cloud connect is disabled. Your cluster boundary is the trust boundary.

Data plane vs control plane



Air-gapped / on-premise

From deployments/:

```
cd deployments
NEXUS_AIRGAP=true NEXUS_CLOUD_CONNECT=false docker compose up -d
curl http://localhost:8090/healthz
```

Helm (AWS EKS, Azure AKS, GCP GKE, or private data centers):

```
helm upgrade --install nexus-shield ./k8s/nexus-shield \
--set global.airgap=true \
--set nexusCloud.connect=false
```

Variable	Default	Meaning
NEXUS_AIRGAP	true	Disable optional cloud egress
NEXUS_CLOUD_CONNECT	false	Control plane off — fully local governance
NEXUS_DATA_PLANE_BOOTSTRAP	true	Local demo tenant for first intercept

Evidence volumes: nexus-enterprise-data, nexus-enterprise-logs.

Custom API addresses & gateway integration

Enterprises point agents at their own base URLs:

- Internal API gateway → https://nexus.internal.company.com/v1/intercept
- Sidecar co-located with agent pods (Kubernetes)
- MCP proxy: bridge SDK in front of tool servers

No requirement to use api.nexusshield.ai when the data plane is self-hosted. See GATEWAY_INTEGRATION.md for LLM router + tool-path wiring.

8. Universal Action Receipts (UAR) & Proof Center Transparency

Canonical UAR fields (production)

Field	Description
receipt_id	Stable receipt identifier (uar_...)
agent_id	Governed agent subject
intent	Declared user / business intent (user_intent in API requests)
proposed_action	Requested tool + params (requested_tool in executive summaries)
intent_divergence	risk_score, violations, divergence metrics (e.g. 96% misalignment)
decision	ALLOW · BLOCK · READ_ONLY · REQUIRE_APPROVAL
execution_state	before_hash, after_hash, execution_status
evidence_hash	SHA-256 seal (evidence_bundle_hash in storage)

Every governed action attempt receives a UAR — not only blocks.

Proof Center — two lanes (do not conflate)

Lane	Representative metrics	What it means
Reproducible benchmark results	127 agents tested · 48,291 tool calls / harness evidence bundles	Open-source harness evaluated trajectory steps (all decision types)
Dangerous-action benchmark subset	3,817 / 3,842 dangerous actions blocked (99.3%)	Attack-scenario block rate — not equal to total evidence count
Deterministic UAR ledger	Per-tenant uar_receipts.jsonl	Your production attempts, decisions, and seals

Public UI copy: **“Every evaluated agent trajectory and governed action attempt produces an immutable SHA-256 evidence bundle.”**

9. End-to-End Demo Scenarios & Workflows

Workflow A — Live interception (executive demo)

1. User intent capture — Operator or planner declares intent (e.g. “Read-only invoice summary”).
2. Tool call proposed — Agent requests `export_customer_database` or MCP equivalent.
3. Interception — `POST /api/v1/action/evaluate` or self-hosted `POST /v1/intercept`.
4. Intent divergence — Policy engine computes risk (e.g. 96% divergence, violations `INTENT_ACTION_DIVERGENCE`).
5. Policy enforcement — `BLOCK`, or `READ_ONLY` / `REQUIRE_APPROVAL` instead of fleet kill.
6. UAR generation — SHA-256 `evidence_hash`, stored receipt, optional SIEM dispatch.
7. Verification — Open `/verify?receipt_hash=...&receipt_id=...` OR `GET /v1/receipts/{id}/verify`.
8. Trust update — `POST /api/v1/agent/trust` adjusts agent reputation / MCP-SEC-SCORE.

Workflow B — CVE preset (Detect & Demonstrate)

```
python scripts/simulate_vulnerability_preset.py --preset cve-2026-critical-zero-day --write-public-json
```

Produces deterministic proof JSON for `/demo` and public verify links — ideal for responsible disclosure rehearsals.

Workflow C — Air-gap proof for procurement

1. Deploy `deployments/docker-compose.yml` in customer lab.
2. Run intercept curl against `localhost:8090`.
3. Show local JSONL ledger + compliance audit log without outbound cloud.
4. Contrast with optional `NEXUS_CLOUD_CONNECT=true` for license-only peering.

Appendix — Key documentation map

Document	Topic
UAR_SCHEMA.md	Canonical receipt fields
DATA_PLANE_AND_CONTROL_PLANE.md	Air-gap architecture
BENCHMARK_VS_ACTION_FIREWALL.md	Harness vs runtime
GATEWAY_INTEGRATION.md	Enterprise gateway placement
COMPLIANCE_READINESS.md	SOC 2 / ISO alignment
DETECT_AND_DEMONSTRATE_PROOF.md	Demo + verify pipeline

Contact: Book a demo · nexusshield.ai